EXPERIMENT 18
First-Order Predicate Logic (FOPC) Parser

Aim

To implement a simple parser for basic first-order predicate logic expressions.

Procedure

1. Define a pattern for predicate expressions.
2. Recognize an optional negation symbol.
3. Extract the predicate name and its arguments.
4. Validate the expression using a regular expression.
5. Display the parsing result.

Program

```
import re

pattern = r"\s*(~?)([A-Za-z][A-Za-z0-9_]*)\(([^()]*)\)\s*"

def parse_fopc(expression):
    match = re.fullmatch(pattern, expression)
    if not match:
        return "Invalid expression"
    negation, predicate, args = match.groups()
    arguments = [item.strip() for item in args.split(",") if item.strip()]
    return {
        "negated": bool(negation),
        "predicate": predicate,
        "arguments": arguments
    }

print(parse_fopc("Likes(Alice, Bob)"))
print(parse_fopc("~Human(Socrates)"))
```

Output

```
OUTPUT — EXPERIMENT 18
{'negated': False, 'predicate': 'Likes', 'arguments': ['Alice', 'Bob']}
{'negated': True, 'predicate': 'Human', 'arguments': ['Socrates']}
```

Result

Thus, a simple FOPC parser extracted predicate names, arguments, and negation.